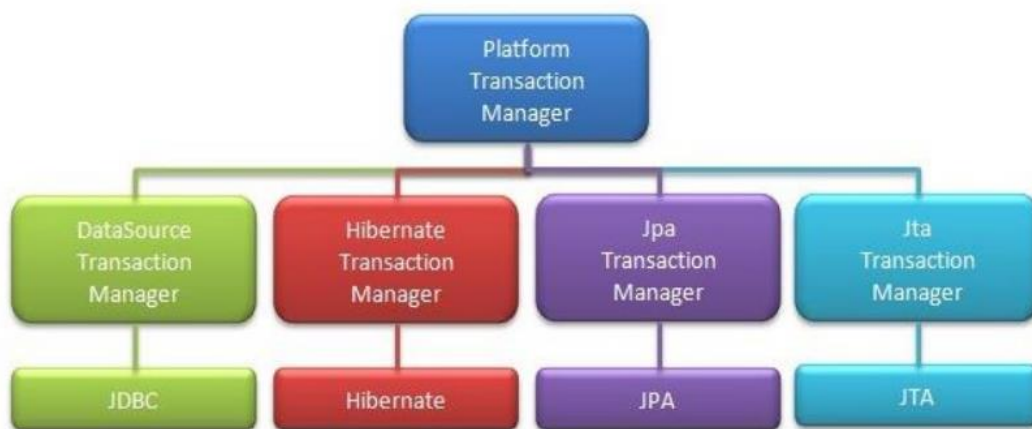


Spring transaction

Spring Transaction은 데이터베이스의 작업을 하나의 단위로 묶어 원자성을 보장하고, 데이터의 일관성을 유지하는 데 사용된다. 트랜잭션이란 일련의 데이터베이스 작업을 하나의 작업 단위로 처리하는 것을 의미하며, 이 작업 단위는 성공적으로 완료되거나 전혀 수행되지 않아야 한다. Spring은 트랜잭션 관리를 위해 다양한 기능과 추상화를 제공한다.

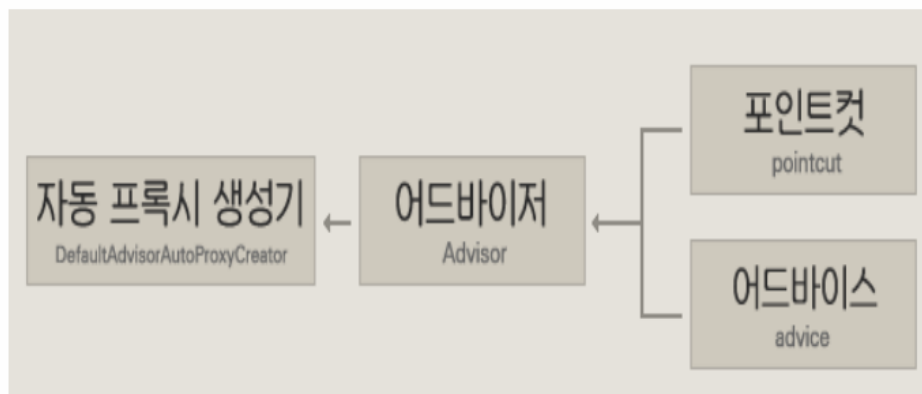
Spring PlatformTransactionManager 설정



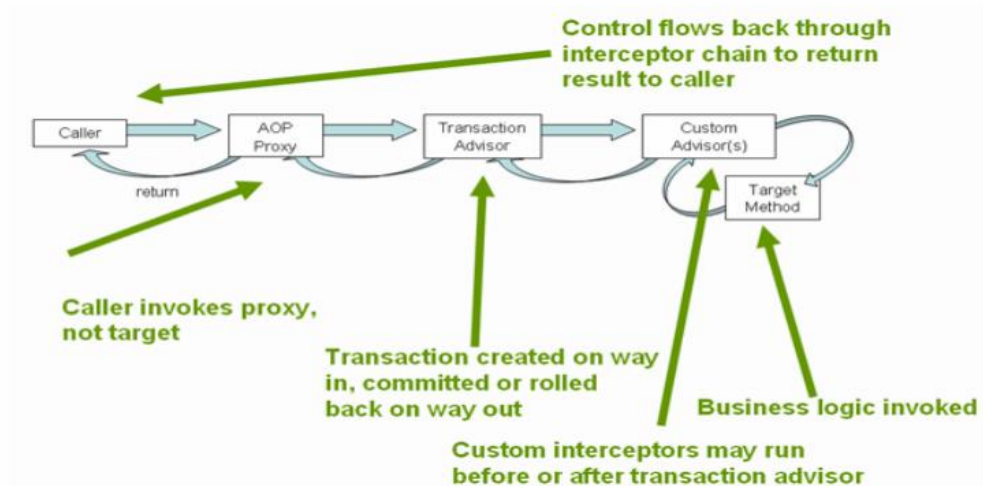
트랜잭션 관리자는 트랜잭션 관리 책임을 JTA나 퍼시스턴스 메커니즘이 제공하는 플랫폼 종속적인 트랜잭션 구현체에 위임한다.

Spring transaction 관리지원

- 트랜잭션을 이해하기 위해 알아야 할 용어



Transaction 동작원리



1. Caller가 target 메소드를 호출하면 먼저 AOP Proxy가 호출된다.
2. Transaction Advisor는 새로운 트랜잭션을 생성한다.
3. Custom interceptor들이 transaction advisor 전 후로 호출된다.
4. Target 비즈니스 메소드가 호출된다.
5. Transaction Advisor는 커밋 또는 롤백을 결정하고 호출자에게 넘긴다.
6. AOP Proxy는 결과를 받아서 Caller에서 넘긴다

주요 개념정리

트랜잭션의 속성 (ACID):

- **Atomicity (원자성)** : 트랜잭션 내의 모든 작업이 성공적으로 완료되거나, 모두 실패하여 데이터베이스의 상태가 변경되지 않도록 보장
- **Consistency (일관성)** : 트랜잭션이 완료되면 데이터베이스의 상태가 일관되게 유지된다. 즉, 트랜잭션 실행 전후에 데이터의 무결성이 보장되어야 한다.
- **Isolation (고립성)** : 동시에 실행되는 트랜잭션은 서로에게 영향을 미치지 않아야 하며, 각 트랜잭션은 독립적으로 수행된다.
- **Durability (지속성)** : 트랜잭션이 성공적으로 완료되면 그 결과가 영구적으로 저장되어야 하며, 시스템 오류가 발생하더라도 결과가 손실되지 않아야 한다.

트랜잭션 전파(Propagation)

Spring에서는 트랜잭션이 어떻게 전파되는지를 설정할 수 있다..

주요 전파 옵션

- A. REQUIRED : 기본값. 이미 트랜잭션이 존재하면 해당 트랜잭션을 사용하고, 없으면 새 트랜잭션을 생성한다.
 - B. REQUIRES_NEW : 항상 새 트랜잭션을 생성하며, 기존 트랜잭션이 있더라도 새로운 트랜잭션을 시작한다.
 - C. NESTED : 현재 트랜잭션 내에서 중첩 트랜잭션을 지원한다. 기존 트랜잭션이 롤백되면 중첩 트랜잭션도 롤백된다.
 - D. SUPPORTS : 기존 트랜잭션이 있으면 그 트랜잭션을 사용하고, 없으면 트랜잭션 없이 실행한다.
 - E. NOT_SUPPORTED : 항상 트랜잭션 없이 실행한다.
 - F. MANDATORY : 반드시 기존 트랜잭션이 존재해야 하며, 없으면 예외를 발생시킨다.
- NEVER : 반드시 트랜잭션 없이 실행하며, 트랜잭션이 있으면 예외를 발생시킨다.

격리 수준(Isolation Level)

격리 수준은 트랜잭션 간의 상호 작용 방식을 정의한다.

주요 격리 수준

- READ_UNCOMMITTED : 다른 트랜잭션의 변경 사항을 볼 수 있다. 가장 낮은 수준으로, 더티 리드(Dirty Read)가 발생할 수 있다.
- READ_COMMITTED : 커밋된 데이터만 읽을 수 있다.. 더티 리드를 방지한다.
- REPEATABLE_READ : 트랜잭션이 실행되는 동안 같은 쿼리를 반복해도 결과가 동일하다. 팬텀 리드(Phantom Read)를 방지한다.
- SERIALIZABLE : 가장 높은 수준으로, 완전한 격리를 제공한다. 다른 트랜잭션이 동시에 실행되는 것을 방지한다.

☞ Spring 에서 트랜잭션 관리

1) 선언적 트랜잭션 관리

Spring에서는 @Transactional 어노테이션을 사용하여 트랜잭션 관리를 선언적으로 설정할 수 있다. 이 방법은 코드의 가독성을 높이고 관리하기 쉽게 해준다

2) 프로그래밍적 트랜잭션 관리

Spring의 PlatformTransactionManager를 사용하여 프로그래밍적으로 트랜잭션을 관리할 수 있다. 이 방법은 좀 더 세밀한 제어가 필요한 경우에 사용한다.

☞ Spring transaction Rollback 규칙

Spring Transaction은 RuntimeException 또는 Error가 발생할 때 기본적으로 롤백한다.

@Transactional의 rollbackFor 속성을 사용하여 특정 Checked 예외에 대해 롤백을 지정할 수 있다.

트랜잭션은 메소드 실행이 끝나거나, @Transactional이 적용된 메소드가 정상적으로 반환될 때 커밋(commit)된다. 이때 예외가 발생하지 않으면 트랜잭션이 커밋되지만, 예외가 발생하면 롤백된다.

계좌이체 시나리오

@Entity
@Builder
@Getter

```

@Setter
@NoArgsConstructor
@AllArgsConstructor
@DynamicUpdate
public class Bank {
    @Id
    private String account;
    private int balance;
}

```

```

4 messages
6 public interface BankRepository extends JpaRepository<Bank, String> {
7
8 }

```

```

10 @SpringBootTest
11 class Step03TransactionApplicationTests {
12     @Autowired
13     private BankRepository bankRepository;
14
15     @Test
16     @DisplayName("계좌등록")
17     void bankInsert() {
18         bankRepository.save(Bank.builder().account("A01").balance(1000).build());
19         bankRepository.save(Bank.builder().account("A02").balance(500).build());
20     }
21 }
22

```

```

6 @Getter
7 @RequiredArgsConstructor
8 public class BasicException extends RuntimeException {
9     private final ErrorCode errorCode;
10 }

```

```

package web.mvc.service;

import jakarta.transaction.Transactional;

```

```
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import web.mvc.domain.Bank;
import web.mvc.dto.RequestTransferDTO;
import web.mvc.exception.BasicException;
import web.mvc.exception.ErrorCode;
import web.mvc.repository.BankRepository;

@Service
@RequiredArgsConstructor
@Transactional
public class BankServiceImpl implements BankService{
    private final BankRepository bankRepository;

    @Override
    public int transfer(RequestTransferDTO requestTransferDTO) throws BasicException {
        //출금 계좌에서 금액 만큼 인출하기
        Bank outBank = bankRepository.findById(requestTransferDTO.getOutAccount())
            .orElseThrow(()->new BasicException(ErrorCode.FAILED_WITHDRAWAL_ACCOUNT));

        outBank.setBalance(outBank.getBalance()-requestTransferDTO.getAmount());

        //입금계좌에 금액 만큼 입금하기
        Bank inBank = bankRepository.findById(requestTransferDTO.getInAccount())
            .orElseThrow(()->new BasicException(ErrorCode.FAILED_DEPOSIT_ACCOUNT));

        inBank.setBalance(inBank.getBalance()+ requestTransferDTO.getAmount());

        //잔액확인하기 - 최대금액이 1000 원을초과하면 예외발생)
        if( inBank.getBalance() > 1000)
            throw new BasicException(ErrorCode.FAILED_MAXIMUM);

        return 1;
    }
}
```

- 출금계좌번호 오류

POST http://localhost:9999/transfer

Body: none form-data x-www-form-urlencoded raw binary JSON

```

1 [JSON]
2 {
3   "inAccount": "A01",
4   "outAccount": "A05", → 출금계좌오류
5   "amount": 100
6 }

```

Status: 200 Time: 427 ms Size: 229 B Save Response

Body: Pretty Raw Preview Visualize JSON

```

1 [JSON]
2 {
3   "status": 200,
4   "msg": "출금계좌번호 오류입니다."
5 }

```

Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?

입금계좌번호 오류

POST http://localhost:9999/transfer

Body: none form-data x-www-form-urlencoded raw binary JSON

```

1 [JSON]
2 {
3   "inAccount": "A05", → 입금계좌번호 오류
4   "outAccount": "A02",
5   "amount": 100
6 }

```

Status: 401 Time: 21 ms Size: 229 B Save Response

Body: Pretty Raw Preview Visualize JSON

```

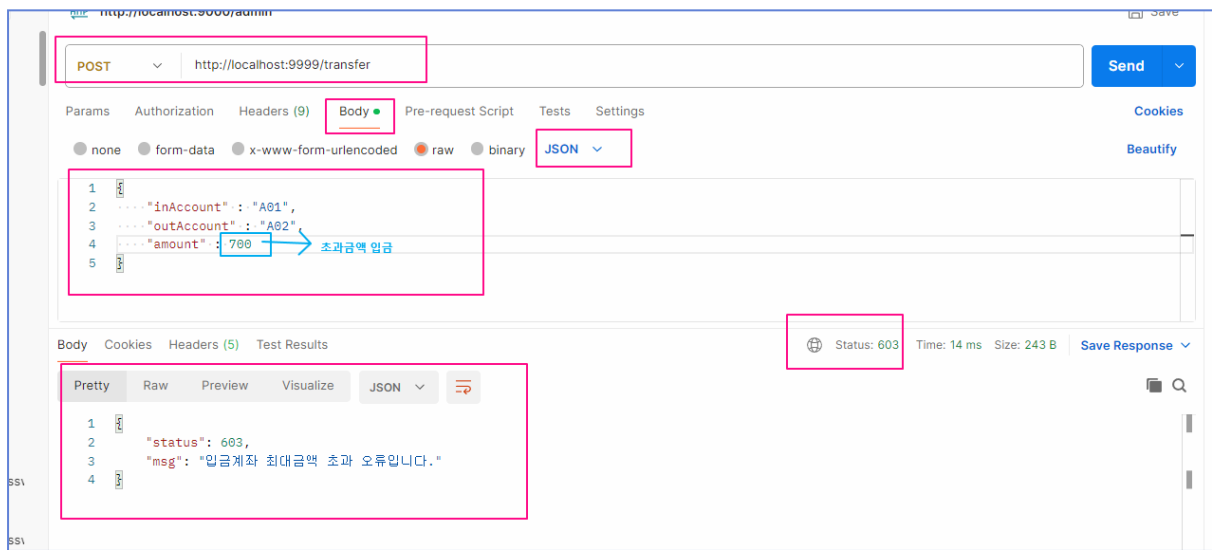
1 [JSON]
2 {
3   "status": 401,
4   "msg": "입금계좌번호 오류입니다."
5 }

```

Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?

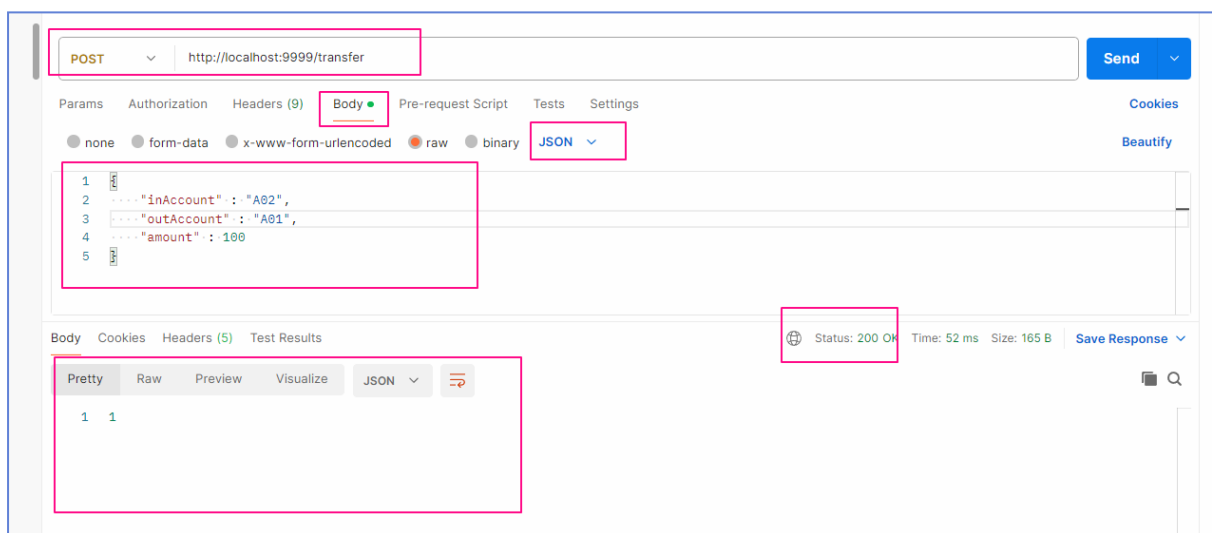
Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?

잔액초과



2024-11-09T20:29:01.010+07:00 INFO 12444 [step02_transaction] INFO 7777 EXEC 0
 Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
 Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
 2024-11-09T20:29:01.027+07:00 ERROR 12444 [step03_transaction] INFO 8888 EXEC 0

성공




```

Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
Hibernate: update bank set balance=? where account=?
Hibernate: update bank set balance=? where account=?

```

아래와 같이 체크예외로 변경 한 후 다시 테스트 해보자!

```

6  @Getter
7  @RequiredArgsConstructor
8  public class BasicException extends Exception {
9      private final ErrorCode errorCode;
10 }
11

```

-입금계좌오류 상황

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:9999/transfer
- Body (JSON):**

```

{
  "inAccount": "A05",
  "outAccount": "A01",
  "amount": 100
}

```
- Status:** 601
- Time:** 29 ms
- Size:** 229 B
- Response Body (JSON):**

```

{
  "status": 601,
  "msg": "입금계좌번호 오류입니다."
}

```

```

Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
Hibernate: update bank set balance=? where account=?

```

→ update가 되었다.

입금계좌 1000원 초과

The screenshot shows a REST client interface. The top bar indicates a POST request to `http://localhost:9999/transfer`. The 'Body' tab is selected, showing a JSON payload: `{ "inAccount": "A02", "outAccount": "A01", "amount": 700 }`. The 'JSON' format is selected. The response section shows a status of 603, with a message: `"msg": "입금계좌 최대금액 초과 오류입니다."`.

```

Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
Hibernate: select b1_0.account,b1_0.balance from bank b1_0 where b1_0.account=?
Hibernate: update bank set balance=? where account=?
Hibernate: update bank set balance=? where account=?

```

→ update가 된다.